Василенко Е.Р. 090303-ПИа-о24

Лабораторная работа №19
Микросервисная архитектура Python-приложений: взаимодействие сервисов и паттерны проектирования

1. Цель работы

Получить практические навыки проектирования и разработки приложений с микросервисной архитектурой: понять отличие монолитной и микросервисной архитектуры, научиться выделять сервисы из предметной области, организовывать синхронное взаимодействие через REST API и асинхронное — через события, реализовать API Gateway, использовать отдельную базу данных для каждого сервиса, реализовать паттерны Retry и Circuit Breaker для обработки временной недоступности сервисов и настроить запуск распределённого приложения.

2. Краткие теоретические сведения

В монолитном приложении все компоненты находятся в одной программе: проще разрабатывать и отлаживать, но сложнее масштабировать отдельные компоненты, и отказ одного компонента влияет на всё приложение. В микросервисной архитектуре приложение состоит из относительно независимых сервисов: каждый сервис имеет чётко определённую ответственность, собственную базу данных (паттерн Database per Service) и REST API, и может развиваться и масштабироваться независимо. Сервисы не обращаются к базам данных друг друга напрямую — только через их API.

Взаимодействие сервисов бывает синхронным (HTTP: сервис вызывает API другого сервиса и ждёт ответ) и асинхронным (через события: сервис публикует событие, другие сервисы-подписчики выполняют действие, не задерживая отправителя). API Gateway — единая точка входа для клиентов: маршрутизация запросов к микросервисам, агрегация, health-check и обработка общих ошибок.

Паттерны отказоустойчивости: Retry — повтор неудачного HTTP-запроса с ограниченным числом попыток, таймаутом и задержкой; Circuit Breaker

(автоматический выключатель) — переход в состояние OPEN при множественных ошибках (запросы временно прекращаются), после паузы — пробный запрос в HALF-OPEN и возврат в CLOSED при успехе или OPEN при ошибке. Событийно-ориентированная архитектура (Event-Driven) — передача событий от одного сервиса другому через брокер; Saga — распределённые транзакции с компенсирующими действиями; CQRS и Transactional Outbox — разделение чтения/записи и надёжная публикация событий.

3. Индивидуальное задание

Вариант 5 — «Ресторан». Микросервисы: menu-service (меню), order-service (заказы), delivery-service (доставка). Событие OrderCreated должно передаваться сервису доставки. Требуется: минимум 3 микросервиса с собственными БД и REST API; API Gateway; паттерны API Gateway, Database per Service, Retry, Circuit Breaker; асинхронное событие OrderCreated → delivery-service; Docker Compose; тесты (минимум 5 unit на сервис, 2 интеграционных, 1 взаимодействия между сервисами, 1 обработки недоступности сервиса — минимум 24 теста).

4. Описание предметной области и обоснование выделения сервисов

Система ресторана разделена на четыре компонента. menu-service отвечает за каталог блюд (название, цена, доступность) — изменяется редко и читается всеми; order-service отвечает за бизнес-процесс оформления заказа — проверку блюда в меню, расчёт стоимости и хранение заказа; delivery-service отвечает за логистику — создание доставки по факту события OrderCreated; gateway — единая точка входа. Границы выбраны по признаку независимой зоны ответственности и частоты изменений: меню редко меняется и часто читается, заказы интенсивно создаются, доставка обрабатывает события асинхронно. Чрезмерное дробление (отдельный сервис на класс) было бы неоправданным — каждый сервис покрывает целую бизнес-функцию.

5. Архитектура приложения

Архитектура системы приведена на рисунке 19.1: клиент обращается к API Gateway, который маршрутизирует запросы к трём микросервисам; order-

service синхронно проверяет блюдо через menu-service и асинхронно публикует событие OrderCreated в delivery-service.

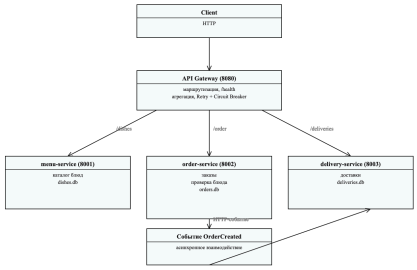


Рисунок 19.1 – Архитектура микросервисной системы

Компонент	Порт	Ответственность	БД
API Gateway	8080	маршрутизация, health-check, агрегация	—
menu-service	8001	каталог блюд	dishes (in-memory)
order-service	8002	заказы, проверка меню, событие	orders.db
delivery-service	8003	доставки (подписчик OrderCreated)	deliveries.db

```
lab_19/
├── menu-service/         # FastAPI, каталог блюд
├── order-service/        # FastAPI, заказы + ServiceClient (Retry, Circuit
Breaker)
├── delivery-service/     # FastAPI, доставки (приём событий)
├── gateway/              # FastAPI, единая точка входа + агрегированный /health
├── docker-compose.yml    # 4 контейнера, сети, переменные окружения
└── README.md
```

6. API микросервисов

Сервис	Endpoint	Назначение	Код
gateway	GET /health	агрегированный health-check	200
gateway	GET /dishes	список блюд → menu-service	200/502
gateway	POST /order	создание заказа → order-service	200/502
menu	GET /dishes	список блюд	200
menu	GET /dishes/{id}	блюдо по ID	200/404
order	POST /orders	создание заказа (проверка меню)	201/503
order	GET /orders	список заказов	200
order	GET /debug/menu-breaker	состояние Circuit Breaker	200
delivery	GET /events?type=OrderCreated	приём события, создание доставки	200/409

Сервис	Endpoint	Назначение	Код
delivery	GET /deliveries	список доставок	200

7. Реализация сервисов

Каждый сервис — отдельное FastAPI-приложение с собственным хранилищем (Database per Service): menu-service держит блюда в памяти, order-service и delivery-service — в собственных SQLite-базах. order-service при создании заказа синхронно вызывает menu-service через ServiceClient, выводя на клиент 503 при недоступности меню, и публикует событие OrderCreated в delivery-service (асинхронно, отказ получателя не отменяет заказ).

Файл: order-service/app/client.py — Retry и Circuit Breaker

```
import logging
import time
from typing import Any, Optional

import httpx

logger = logging.getLogger("ms-client")


class CircuitOpenError(Exception):
    """Состояние OPEN: запросы к сервису временно запрещены."""


class CircuitBreaker:
    """Автоматический выключатель: CLOSED → OPEN → HALF-OPEN → CLOSED/OPEN."""

    def __init__(self, failure_threshold: int = 3, open_timeout: float = 2.0):
        self.failure_threshold = failure_threshold
        self.open_timeout = open_timeout
        self.failures = 0
        self.state = "CLOSED"
        self.opened_at = 0.0

    def allow(self) -> bool:
        if self.state == "OPEN":
            if time.monotonic() - self.opened_at >= self.open_timeout:
                self.state = "HALF-OPEN"
                return True
            return False
        return True

    def success(self) -> None:
        self.failures = 0
        self.state = "CLOSED"

    def failure(self) -> None:
        self.failures += 1
        if self.state == "HALF-OPEN" or self.failures >= self.failure_threshold:
            self.state = "OPEN"
            self.opened_at = time.monotonic()

    def __repr__(self) -> str:
        return f"CircuitBreaker(state={self.state}, failures={self.failures})"


class ServiceClient:
    """HTTP-клиент с паттернами Retry и Circuit Breaker."""

    RETRYABLE_STATUS = {503, 504}
```

```
    def __init__(self, base_url: str, retries: int = 3, backoff: float = 0.15,
                 breaker: Optional[CircuitBreaker] = None):
        self.base_url = base_url.rstrip("/")
        self.retries = retries
        self.backoff = backoff
        self.breaker = breaker or CircuitBreaker()

    def _url(self, path: str) -> str:
        return f"{self.base_url}/{path.lstrip('/')}"
```

Файл: order-service/app/main.py — создание заказа

```
    if row is None:
        raise HTTPException(status_code=404, detail="Order not found")
    return Order(id=row[0], dish_id=row[1], client_name=row[2], total=row[3],
                 status=row[4], created_at=row[5], dish_name="?")


@app.get("/create")
def create_order_get(dish_id: int, client_name: str = "client") -> Order:
    """GET-обёртка для gwateway-агрегации (имитация POST через gateway)."""
    return create_order(OrderCreate(dish_id=dish_id, client_name=client_name))


@app.post("/orders", status_code=201)
def create_order(payload: OrderCreate) -> Order:
    # синхронное взаимодействие: проверить блюдо в menu-service
    try:
        dish = menu_client.get(f"/dishes/{payload.dish_id}")
    except Exception as error:
        raise HTTPException(status_code=503, detail=f"Menu service unavailable:
{error}")

    connection = _connect()
    cursor = connection.execute(
        "INSERT INTO orders (dish_id, client_name, total, status, created_at) "
        "VALUES (?, ?, ?, ?, ?)",
        (payload.dish_id, payload.client_name, dish["price"], "created",
         datetime.now().isoformat(timespec="seconds")),
    )
    connection.commit()
    order_id = cursor.lastrowid
    row = connection.execute("SELECT * FROM orders WHERE id = ?",
(order_id,)).fetchone()
    connection.close()

    order = Order(id=order_id, dish_id=payload.dish_id, dish_name=dish["name"],
                  client_name=payload.client_name, total=dish["price"],
                  status=row[4], created_at=row[5])

    # асинхронное событие OrderCreated -> delivery-service
    try:
        delivery_service.get(
            f"/events?type=OrderCreated&order_id={order.id}&dish={dish['name']}"
        )
```

8. API Gateway

Gateway маршрутизирует запросы к микросервисам и предоставляет агрегированный health-check: проверяет GET /health каждого сервиса и возвращает общее состояние (healthy/degraded) с состоянием каждого сервиса. Клиенту не нужно знать адреса микросервисов — единая точка входа на порту 8080. В реализации gateway также используется ServiceClient с паттернами Retry и Circuit Breaker.

Файл: gateway/app/main.py — health и маршрутизация

```
from fastapi import FastAPI, HTTPException

from app.client import ServiceClient

app = FastAPI(title="API Gateway")

MENU_URL = "http://127.0.0.1:8001"
ORDER_URL = "http://127.0.0.1:8002"
DELIVERY_URL = "http://127.0.0.1:8003"

menu = ServiceClient(MENU_URL, retries=2, backoff=0.05)
orders = ServiceClient(ORDER_URL, retries=2, backoff=0.05)
delivery = ServiceClient(DELIVERY_URL, retries=1)


@app.get("/")
def root() -> dict:
    return {
        "service": "API Gateway",
        "endpoints": ["/health", "/dishes", "/orders", "/deliveries", "/order"],
    }


@app.get("/health")
def health() -> dict:
    """Агрегированный health-check по всем сервисам."""
    statuses = {}
    for name, client in [("menu", menu), ("orders", orders), ("delivery",
delivery)]:
        try:
            client.get("/health")
            statuses[name] = "ok"
        except Exception:
            statuses[name] = "unavailable"
    overall = "healthy" if all(v == "ok" for v in statuses.values()) else
"degraded"
    return {"status": overall, "services": statuses}


@app.get("/dishes")
def proxy_dishes() -> list:
    try:
        return menu.get("/dishes")
    except Exception as error:
        raise HTTPException(status_code=502, detail=str(error))
```

9. Реализованные паттерны

API Gateway — единая точка входа; Database per Service — у каждого сервиса своё хранилище; Retry — повтор временных ошибок (503/504, отсутствие соединения) с ограниченным числом попыток (по умолчанию 3) и растущей задержкой между попытками; Circuit Breaker — при пороге ошибок (3) переход CLOSED → OPEN, при котором запросы выполняются без обращения к сервису, после тайм-аута — пробный запрос HALF-OPEN: при успехе возврат в CLOSED, при ошибке — снова OPEN. Асинхронное взаимодействие — событие OrderCreated, передаваемое из order-service в

delivery-service через событийный endpoint (имитация брокера сообщений); доставка создаётся подписчиком.

10. Docker Compose

Файл docker-compose.yml описывает четыре сервиса (gateway, menu-service, order-service, delivery-service), их порты, переменные окружения с адресами внутренних хостов (например, MENU_SERVICE_URL=http://menu-service:8001) и зависимости (gateway зависит от всех сервисов, order-service — от menu и delivery). Каждый микросервис упакован в собственный Dockerfile на базе python:3.12-slim. Запуск системы выполняется одной командой docker compose up --build.

11. Демонстрация работы

Живая демонстрация системы (четыре uvicorn-процесса): агрегированный health-check gateway, каталог блюд, создание заказа с проверкой меню и доставка после события OrderCreated (рисунок 19.2).



Рисунок 19.2 – Демонстрация: gateway, меню, заказ, доставка

Экспериментальная проверка отказоустойчивости (рисунок 19.2, нижняя часть): после остановки menu-service gateway возвращает degraded (menu unavailable); заказ через order-service сначала повторяет запросы (Retry), затем выключатель переходит в OPEN (failures=3, state=OPEN); после восстановления menu-service успешный заказ переводит выключатель обратно в CLOSED (failures=0).

12. Тестирование

Написано 25 автоматических тестов: menu-service — 5 (health, список, получение, 404, доступность); order-service — 10 (unit-тесты сервиса с Mock-репозиторием меню, недоступность меню со статусом 503, тесты Circuit Breaker: CLOSED/OPEN/HALF-OPEN, тесты ServiceClient: Retry до успеха и до открытия выключателя, проверка вызовов, 404, health); delivery-service — 4 (приём события, дубликат 409, список доставок, health); gateway — 6 (маршрутизация, здоровье всех/частичное, создание заказа, корневой endpoint). Для изоляции unit-тестов межсервисное взаимодействие заменяется Mock (patch). Результаты pytest приведены на рисунке 19.3.

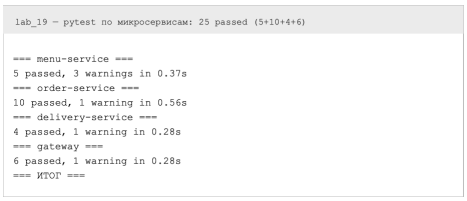


Рисунок 19.3 – Результаты pytest по всем микросервисам (25 passed)

13. Вывод

В ходе лабораторной работы разработано распределённое приложение «Ресторан» из трёх микросервисов и API Gateway: каждый сервис имеет чёткую зону ответственности, собственное хранилище (Database per Service) и REST API; синхронное взаимодействие реализовано через HTTP (order-service проверяет блюдо в menu-service), асинхронное — через событие OrderCreated, принимаемое delivery-service.

Реализованы обязательные паттерны: API Gateway с агрегированным health-check, Retry повторных запросов и Circuit Breaker с состояниями CLOSED/OPEN/HALF-OPEN. Экспериментальная проверка подтвердила: при недоступности menu-service заказы после повторов завершаются ошибкой, выключатель переходит в OPEN, а после восстановления сервиса система возвращается в нормальный режим. 25 автоматических тестов покрывают

каждый сервис, включая взаимодействие между сервисами и обработку их недоступности. Полученные навыки проектирования границ сервисов, организации их взаимодействия и отказоустойчивости являются основой для построения масштабируемых распределённых систем.